Developer’s Manual

CS3383 Spring 2026

Environment Setup

E.T. Pizzeria is developed in Unity Version 6.3.10f1

Installing Unity

1. Go to Unity's Download Page and click "Download Installer for Windows". A UnityDownloadAssistant-6.3.exe file should be downloaded to your "Downloads" folder.
2. Open the downloaded installer.
3. Accept the license and terms and click Next.
4. Select the components you would like to be installed with Unity and click "Next".
    a. Note: If you ever want to change the components, you can re-run the installer.
5. You can change where you want Unity installed or leave the default option and click "Next".
6. Depending on the components you selected, you may see additional prompts before installing.
7. Follow the prompts and click "Install". Installing Unity may take some time.
8. After the installation is finished, Unity will be installed on your computer.
9. Go into Unity Hub and click "Installs"
10. Ensure version 6.3.10f1 is installed, and the development process may begin.
    a. If version 6.3.10f1 is not installed, continue with the following steps
11. Locate the desired version and click Install.
12. After the installation is complete, the Unity editor version is installed.

Overview of E.T. Pizzeria

The context diagram and the existing class diagrams are the basis of the current code structure for the core system E.T. Pizzeria. The Player class handles taking orders, applying toppings, cutting pizza, and baking pizza. This directly interacts with the different stations throughout the game that reached one after the other when that station is complete. The score at the end is calculated by taking accuracy from the different stations and calculated with what the order specified.

Oral Exam Material

Prefabs

"A **Prefab** asset type that allows you to store a GameObject complete with components and properties." Acting like a template, prefabs allow you to create multiple instances of a GameObject. Prefabs also allow easy editing of components and properties, as edits are reflected in all instances of the prefab.

Creating Prefabs

There are two methods for creating a prefab: dragging from the Hierarchy or using scripts.

Dragging from Hierarchy to Project Window

1. Set up your GameObject (and any children and components of the prefab) in the **Hierarchy window**.
2. Drag the **GameObject** into the P**roject window**, preferably into a prefab folder. This creates a new prefab asset and turns the original object into an instance of that prefab.

Creating Prefabs via Scripts

1. Use "**using UnityEditor;**" namespace inside an editor script. (Must be created in an editor script)
2. Add the components and configure them as needed.
3. Ensure the path is unique and inside the **Assets** folder.
4. Save the GameObject as a prefab asset using "**PrefabUtility.SaveAsPrefabAsset();"**

Using Prefabs

To create an instance of a prefab, drag and drop the prefab file into the **Scene** or the **Hierarchy**. This will create a new instance. There is no limit to the number of instances a prefab can have.

Editing Prefabs

1. Click on the **prefab asset** in the **Project window** to open it in isolation.
2. Open the Inspector or **use the arrow in the Hierarchy** to edit it in the context of the scene.
3. The changes will apply to all instances of the prefab automatically.
4. **Overrides** can be used to make changes to a single instance in the scene without affecting the base asset.
5. Use **Prefab Variants** to make specialized versions of a prefab, leaving the base asset unchanged.

Prefab Documentation (Main topic for prefabs during oral exam)

Documenting prefabs helps users quickly understand what the prefab does, how it works, and how to integrate it into their own projects. Clear documentation provides the user/buyer with an understanding of functionality, setup, and efficient reuse and easier implementation of the prefab. The following information is recommended to be included in prefab documentation:

Prefab Documentation Template

1. Prefab Name
    - Name of the prefab as it appears in unity or asset store.
2. Author / Creator
3. Version
    - Prefab version (*e.g. Version 1*)
4. Price / Rating / Media
    - Price of the prefab in the asset store.
    - User rating of the prefab. (*e.g. 4/5 stars*).
    - Include a video Snippet / GIF to preview the prefab in action.
5. Description
    - An overview of the prefab, its purpose, and how it can benefit the user.
6. Features
    - List key Features or functionalities of the prefab.
7. Components
    - List all major components attached to the prefab. Include scripts, physics, rendering, and colliders.

8. Setup Instructions
    - Step-by-step instructions to integrate the prefab.
9. Requirements
    - Unity Version (*e.g. Unity 2026.6.32f1 or later*).
    - Dependencies / Plugins (*e.g. Cinemachine, DOTween*).
10. Acknowledgements / Sources(*Optional*)
    - Credit any assets, sprites or sounds used.
11. Notes / Tips (*Optional*)
    - Any additional guidance, tips or warnings.

Links to well documented prefabs:

https://eodl-webv-1.eo.uidaho.edu/drbc/cs383/OralExam/prefab_slime.html

https://eodl-webv 1.eo.uidaho.edu/drbc/cs383/OralExam/prefab_AudioManager.html

https://eodl-webv-1.eo.uidaho.edu/drbc/cs383/OralExam/prefab_pong.html

Patterns

Design Patterns - Patterns

A **design pattern** is a reusable solution to a common software design problem. It provides developers with a template for writing code that is more efficient, maintainable, and adaptable to specific situations.

Design Patterns have two major sources:

- GRASP: General Responsibility Assignment Software Patterns
- GOF: Gang of Four

References

- https://docs.unity3d.com/540/Documentation/Manual/Prefabs.html
- https://docs.unity3d.com/6000.3/Documentation/Manual/CreatingPrefabs.html#create-an-instance-of-a-prefab
- https://sourcemaking.com/design_patterns